

# Taller 5

Juan Pablo Sosa Camargo

## Table of contents

|          |                                                                                   |          |
|----------|-----------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Tipo Razón Preprocesamiento mínimo</b>                                         | <b>1</b> |
| 1.0.1    | Tipo de dato de la fuente . . . . .                                               | 4        |
| 1.0.2    | ¿Por qué <code>fromJSON()</code> no produce directamente un tibble plano? . . . . | 4        |
| 1.0.3    | Resultados . . . . .                                                              | 6        |
| 1.0.4    | Interpretación . . . . .                                                          | 24       |

1. Ejercicio 1: clasificación de tipos de datos

## 1 Tipo Razón Preprocesamiento mínimo

1. CSV DANE Estructurada Datos organizados en filas y columnas. Leer CSV y revisar tipos/nulos.
  2. JSON API Semi-estructurada Tiene campos y estructura mediante claves, pero puede ser anidada. Convertir JSON a tabla y desanidar campos.
  3. Audio judicial No estructurada Es información sonora sin filas/columnas definidas. Transcribir el audio a texto y estructurarlo.
  4. Factura XML Semi-estructurada Usa etiquetas y jerarquías para organizar los datos. Parsear XML y extraer campos a columnas.
  5. Tabla HTML Semi-estructurada Tiene estructura de tabla, pero proviene de código HTML. Extraer la tabla y convertirla a tibble.
2. Ejercicio 2: JSON anidado a DataFrame
  - 1.

```
library(jsonlite); library(dplyr); library(tibble)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
url <- "https://jsonplaceholder.typicode.com/users"
raw <- fromJSON(url)
str(raw)
```

```
'data.frame': 10 obs. of 8 variables:
 $ id      : int  1 2 3 4 5 6 7 8 9 10
 $ name    : chr  "Leanne Graham" "Ervin Howell" "Clementine Bauch" "Patricia Lebsack" ...
 $ username: chr  "Bret" "Antonette" "Samantha" "Karianne" ...
 $ email   : chr  "Sincere@april.biz" "Shanna@melissa.tv" "Nathan@yesenia.net" "Julianne.OConor" ...
 $ address : 'data.frame': 10 obs. of 5 variables:
 ..$ street : chr  "Kulas Light" "Victor Plains" "Douglas Extension" "Hoeger Mall" ...
 ..$ suite  : chr  "Apt. 556" "Suite 879" "Suite 847" "Apt. 692" ...
 ..$ city   : chr  "Gwenborough" "Wisokyburgh" "McKenziehaven" "South Elvis" ...
 ..$ zipcode: chr  "92998-3874" "90566-7771" "59590-4157" "53919-4257" ...
 ..$ geo    : 'data.frame': 10 obs. of 2 variables:
 .. ..$ lat: chr  "-37.3159" "-43.9509" "-68.6102" "29.4572" ...
 .. ..$ lng: chr  "81.1496" "-34.4618" "-47.0653" "-164.2990" ...
 $ phone   : chr  "1-770-736-8031 x56442" "010-692-6593 x09125" "1-463-123-4447" "493-170-9631" ...
 $ website : chr  "hildegard.org" "anastasia.net" "ramiro.info" "kale.biz" ...
 $ company : 'data.frame': 10 obs. of 3 variables:
 ..$ name      : chr  "Romaguera-Crona" "Deckow-Crist" "Romaguera-Jacobson" "Robel-Corkery" ...
 ..$ catchPhrase: chr  "Multi-layered client-server neural-net" "Proactive didactic contingency" ...
 ..$ bs        : chr  "harness real-time e-markets" "synergize scalable supply-chains" "e-enablement" ...
```

Las listas anidadas son: address y company son columnas anidadas; dentro de address. geo está doblemente anidada.

2.

```
library(tidyverse)
```

Warning: package 'tidyverse' was built under R version 4.5.2

Warning: package 'ggplot2' was built under R version 4.5.2

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
v forcats   1.0.1      v readr      2.1.5
```

```
v ggplot2   4.0.1      v stringr    1.5.2
```

```
v lubridate 1.9.4      v tidyr      1.3.1
```

```
v purrr     1.1.0
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
```

```
x purrr::flatten() masks jsonlite::flatten()
```

```
x dplyr::lag()     masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
usuarios <- raw %>%
  unnest(address) %>%
  unnest(geo) %>%
  unnest(company, names_sep = "_") %>%
  select(
    name,
    email,
    city,
    lat,
    lng,
    company_name
  ) %>%
  rename(
    address.city = city,
    address.geo.lat = lat,
    address.geo.lng = lng,
    company.name = company_name
  )
```

```
usuarios
```

```
# A tibble: 10 x 6
```

|    | name           | email | address.city | address.geo.lat | address.geo.lng | company.name |
|----|----------------|-------|--------------|-----------------|-----------------|--------------|
|    | <chr>          | <chr> | <chr>        | <chr>           | <chr>           | <chr>        |
| 1  | Leanne Graham  | Sinc~ | Gwenborough  | -37.3159        | 81.1496         | Romaguera-C~ |
| 2  | Ervin Howell   | Shan~ | Wisokyburgh  | -43.9509        | -34.4618        | Deckow-Crist |
| 3  | Clementine B~  | Nath~ | McKenziehav~ | -68.6102        | -47.0653        | Romaguera-J~ |
| 4  | Patricia Leeb~ | Juli~ | South Elvis  | 29.4572         | -164.2990       | Robel-Corke~ |
| 5  | Chelsey Diet~  | Luci~ | Roscoeview   | -31.8129        | 62.5342         | Keebler LLC  |
| 6  | Mrs. Dennis ~  | Karl~ | South Chris~ | -71.4197        | 71.7478         | Considine-L~ |
| 7  | Kurtis Weiss~  | Tell~ | Howemouth    | 24.8918         | 21.8984         | Johns Group  |
| 8  | Nicholas Run~  | Sher~ | Aliyaview    | -14.3990        | -120.7677       | Abernathy G~ |
| 9  | Glenna Reich~  | Chai~ | Bartholomeb~ | 24.6463         | -168.8889       | Yost and So~ |
| 10 | Clementina D~  | Rey.~ | Lebsackbury  | -38.2386        | 57.2232         | Hoeger LLC   |

### 1.0.1 Tipo de dato de la fuente

La fuente es semi-estructurada (JSON), ya que organiza la información mediante claves y valores, pero permite tener datos anidados.

### 1.0.2 ¿Por qué fromJSON() no produce directamente un tibble plano?

fromJSON() conserva la estructura original del archivo JSON. En este caso, existen objetos anidados como address, geo y company, por lo que R los convierte en data.frame dentro de otros data.frame.

Por esta razón, es necesario aplanar o desanidar la información para obtener un tibble plano con una fila por usuario.

### 3. Ejercicio 3: matriz de diseño

```
library(Matrix)
```

Attaching package: 'Matrix'

The following objects are masked from 'package:tidyr':

expand, pack, unpack

```
# 1. Cargar los datos
titanic <- read.csv(
  "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
)
library(Matrix)

# Seleccionar variables y eliminar NA
datos <- titanic[, c("Age", "Fare", "SibSp", "Parch")]
datos <- na.omit(datos)

# Construir X centrada y escalada
X <- scale(datos)

# Dimensiones de X
dim(X)
```

```
[1] 714    4
```

```
# Memoria de X
object.size(X)
```

```
70504 bytes
```

```
# Calcular X'X
XTX <- t(X) %*% X
XTX
```

|       | Age        | Fare      | SibSp      | Parch     |
|-------|------------|-----------|------------|-----------|
| Age   | 713.00000  | 68.49555  | -219.77994 | -134.8420 |
| Fare  | 68.49555   | 713.00000 | 98.62843   | 146.2498  |
| SibSp | -219.77994 | 98.62843  | 713.00000  | 273.6636  |
| Parch | -134.84203 | 146.24976 | 273.66356  | 713.0000  |

```
# Diagonal de X'X
diag(XTX)
```

| Age | Fare | SibSp | Parch |
|-----|------|-------|-------|
| 713 | 713  | 713   | 713   |

```
# Crear versión dispersa
X_sparse <- Matrix(X, sparse = TRUE)

# Comparar memoria
object.size(X)
```

70504 bytes

```
object.size(X_sparse)
```

81848 bytes

```
# Proporción de ceros
mean(X == 0)
```

[1] 0

### 1.0.3 Resultados

Se seleccionaron las variables **Age**, **Fare**, **SibSp** y **Parch** y se eliminaron las filas que contenían valores NA. Posteriormente se construyó la matriz **X** centrada y escalada mediante **scale()**.

La matriz  $XTX = t(X) \%*\% X$  corresponde a la matriz de productos cruzados. Como las variables están estandarizadas, los valores de la diagonal representan la suma de cuadrados de cada variable estandarizada.

La matriz dispersa se construyó mediante **Matrix(X, sparse = TRUE)**. En este caso, el formato disperso no resulta especialmente conveniente porque la matriz contiene muy pocos valores iguales a cero. Las matrices dispersas son más útiles cuando una gran proporción de sus elementos son cero.

Para representar las relaciones familiares entre pasajeros utilizaría un **grafo**, donde cada pasajero sería un nodo y las relaciones familiares serían las conexiones entre los nodos.

Ejercicio 4: concentración de distancias

```
library(tidyverse)

# =====
# EJERCICIO 4: MALDICIÓN DE LA DIMENSIONALIDAD
# =====
```

```

set.seed(42)

# Dimensiones solicitadas
p_values <- c(1, 2, 5, 10, 50, 100, 500, 1000, 5000)

# -----
# 1. DISTANCIA EUCLIDIANA
# -----

resultados_euc <- map_dfr(p_values, function(p) {

  # 500 puntos uniformes en [0,1]^p
  datos <- matrix(
    runif(500 * p),
    nrow = 500,
    ncol = p
  )

  # 200 pares de puntos
  pares <- matrix(
    sample(1:500, 400, replace = TRUE),
    ncol = 2
  )

  # Distancia euclidiana
  A <- datos[pares[, 1], , drop = FALSE]
  B <- datos[pares[, 2], , drop = FALSE]

  distancias <- sqrt(rowSums((A - B)^2))

  # Media y coeficiente de variación
  tibble(
    p = p,
    distancia_media = mean(distancias),
    CV = sd(distancias) / mean(distancias)
  )
})

# Mostrar resultados
print(resultados_euc)

```

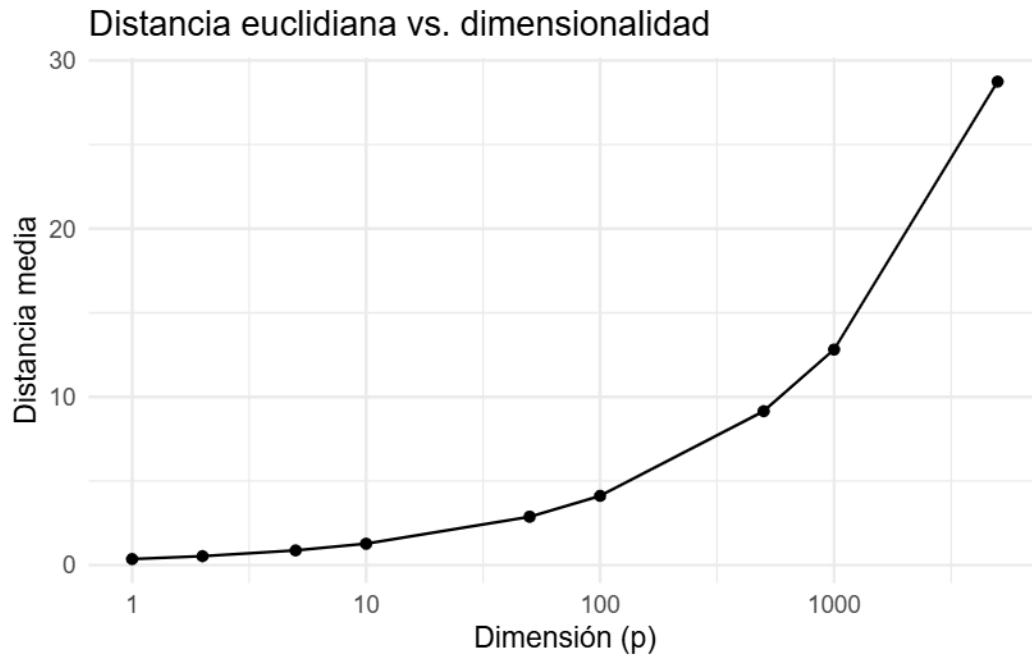
```
# A tibble: 9 x 3
      p distancia_media    CV
  <dbl>      <dbl> <dbl>
1     1      0.362 0.701
2     2      0.530 0.462
3     5      0.873 0.290
4    10      1.27 0.192
5    50      2.88 0.0917
6   100      4.11 0.0625
7   500      9.15 0.0252
8  1000     12.8 0.0731
9  5000     28.7 0.0716
```

```
# -----
# 2. GRÁFICO: DISTANCIA MEDIA VS p
# -----

grafico_media <- ggplot(
  resultados_euc,
  aes(x = p, y = distancia_media)
) +
  geom_line() +
  geom_point() +
  scale_x_log10() +
  labs(
    title = "Distancia euclidiana vs. dimensionalidad",
    x = "Dimensión (p)",
    y = "Distancia media"
  ) +
  theme_minimal()

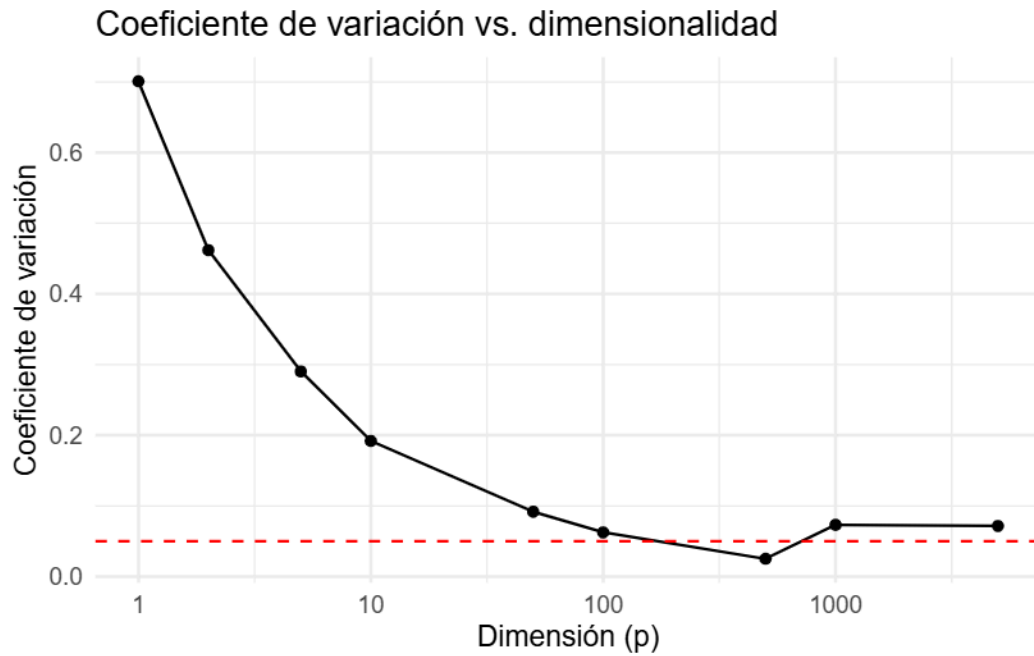
print(grafico_media)
```





```
# -----  
# 3. GRÁFICO: CV VS p  
# -----  
  
grafico_cv <- ggplot(  
  resultados_euc,  
  aes(x = p, y = CV)  
) +  
  geom_line() +  
  geom_point() +  
  geom_hline(  
    yintercept = 0.05,  
    linetype = "dashed",  
    color = "red"  
  ) +  
  scale_x_log10() +  
  labs(  
    title = "Coeficiente de variación vs. dimensionalidad",  
    x = "Dimensión (p)",  
    y = "Coeficiente de variación"  
  ) +  
  theme_minimal()
```

```
print(grafico_cv)
```



```
# -----  
# 4. ¿A PARTIR DE QUÉ p EL CV ES MENOR AL 5%?  
# -----
```

```
resultado_5 <- resultados_euc %>%  
  filter(CV < 0.05) %>%  
  slice(1)  
  
print(resultado_5)
```

```
# A tibble: 1 x 3  
  p distancia_media    CV  
  <dbl>         <dbl> <dbl>  
1   500           9.15 0.0252
```

```
# -----  
# 5. DISTANCIA COSENO  
# -----
```

```

set.seed(42)

resultados_cos <- map_dfr(p_values, function(p) {

  # 500 puntos uniformes en [0,1]^p
  datos <- matrix(
    runif(500 * p),
    nrow = 500,
    ncol = p
  )

  # 200 pares
  pares <- matrix(
    sample(1:500, 400, replace = TRUE),
    ncol = 2
  )

  A <- datos[pares[, 1], , drop = FALSE]
  B <- datos[pares[, 2], , drop = FALSE]

  # Similitud coseno
  producto <- rowSums(A * B)

  norma_A <- sqrt(rowSums(A^2))
  norma_B <- sqrt(rowSums(B^2))

  similitud <- producto / (norma_A * norma_B)

  # Distancia coseno
  distancias <- 1 - similitud

  tibble(
    p = p,
    distancia_media = mean(distancias),
    CV = sd(distancias) / mean(distancias)
  )
})

# Mostrar resultados
print(resultados_cos)

```

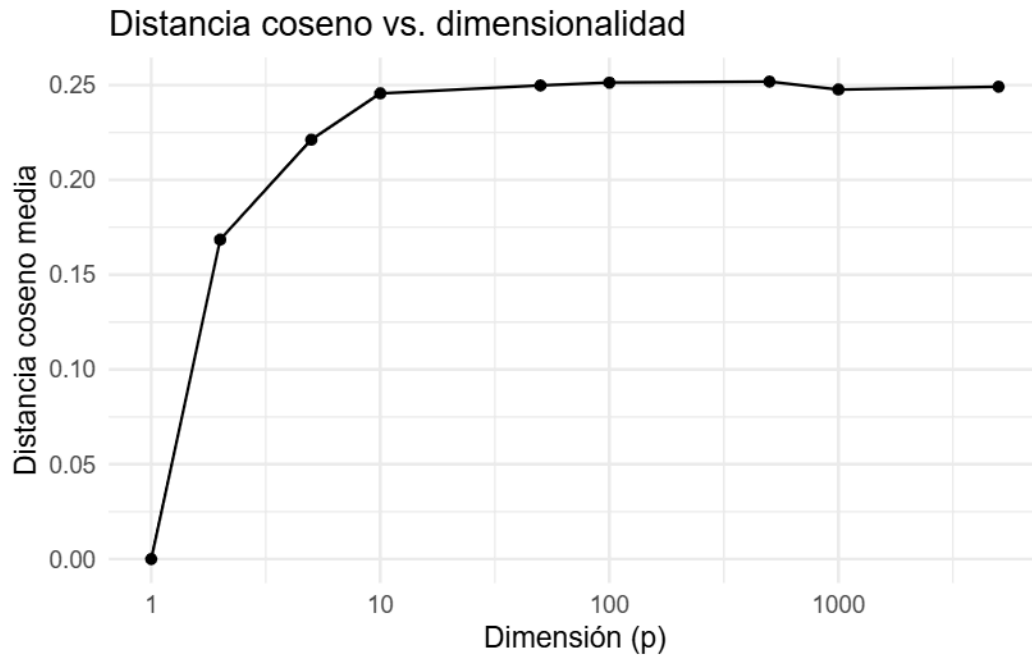
```
# A tibble: 9 x 3
```

|   | p     | distancia_media | CV     |
|---|-------|-----------------|--------|
|   | <dbl> | <dbl>           | <dbl>  |
| 1 | 1     | 0               | NaN    |
| 2 | 2     | 0.168           | 1.17   |
| 3 | 5     | 0.221           | 0.572  |
| 4 | 10    | 0.246           | 0.393  |
| 5 | 50    | 0.250           | 0.194  |
| 6 | 100   | 0.251           | 0.135  |
| 7 | 500   | 0.252           | 0.0548 |
| 8 | 1000  | 0.248           | 0.0795 |
| 9 | 5000  | 0.249           | 0.0731 |

```
# -----
# 6. GRÁFICO: DISTANCIA COSENO
# -----

grafico_cos_media <- ggplot(
  resultados_cos,
  aes(x = p, y = distancia_media)
) +
  geom_line() +
  geom_point() +
  scale_x_log10() +
  labs(
    title = "Distancia coseno vs. dimensionalidad",
    x = "Dimensión (p)",
    y = "Distancia coseno media"
  ) +
  theme_minimal()

print(grafico_cos_media)
```



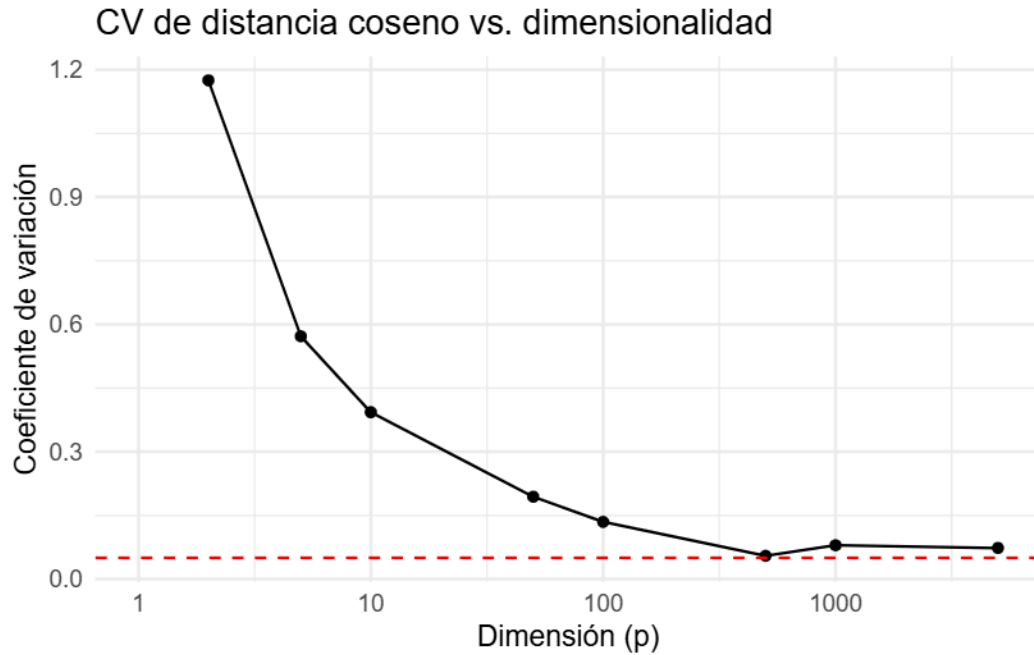
```
# -----
# 7. GRÁFICO: CV DE DISTANCIA COSENO
# -----

grafico_cos_cv <- ggplot(
  resultados_cos,
  aes(x = p, y = CV)
) +
  geom_line() +
  geom_point() +
  geom_hline(
    yintercept = 0.05,
    linetype = "dashed",
    color = "red"
  ) +
  scale_x_log10() +
  labs(
    title = "CV de distancia coseno vs. dimensionalidad",
    x = "Dimensión (p)",
    y = "Coeficiente de variación"
  ) +
  theme_minimal()
```

```
print(grafico_cos_cv)
```

Warning: Removed 1 row containing missing values or values outside the scale range (``geom_line()``).

Warning: Removed 1 row containing missing values or values outside the scale range (``geom_point()``).



```
# -----  
# 8. ¿A PARTIR DE QUÉ p EL CV COSENO ES MENOR AL 5%?  
# -----  
  
resultado_5_cos <- resultados_cos %>%  
  filter(CV < 0.05) %>%  
  slice(1)  
  
print(resultado_5_cos)
```

```
# A tibble: 0 x 3  
# i 3 variables: p <dbl>, distancia_media <dbl>, CV <dbl>
```

```
# -----
# 9. COMPARACIÓN FINAL EUCLIDIANA VS COSENO
# -----

comparacion <- resultados_euc %>%
  select(p, CV_euclidiana = CV) %>%
  left_join(
    resultados_cos %>%
      select(p, CV_coseno = CV),
    by = "p"
  )

print(comparacion)
```

```
# A tibble: 9 x 3
      p CV_euclidiana CV_coseno
  <dbl>      <dbl>      <dbl>
1     1      0.701      NaN
2     2      0.462      1.17
3     5      0.290      0.572
4    10      0.192      0.393
5    50      0.0917     0.194
6   100      0.0625     0.135
7   500      0.0252     0.0548
8  1000      0.0731     0.0795
9  5000      0.0716     0.0731
```

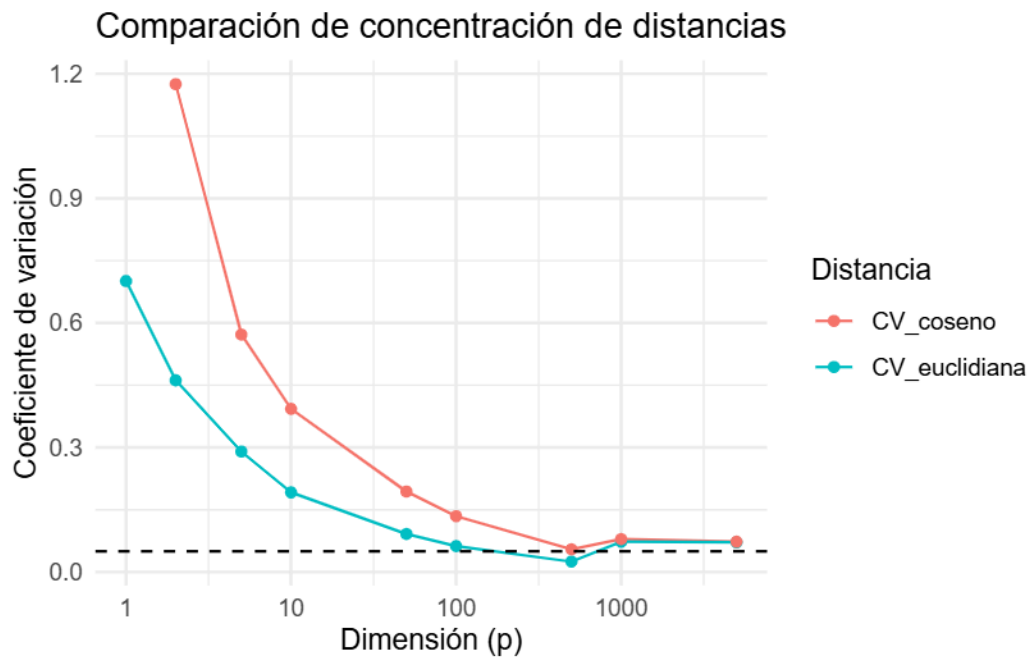
```
# Gráfico comparativo de CV
comparacion_larga <- comparacion %>%
  pivot_longer(
    cols = c(CV_euclidiana, CV_coseno),
    names_to = "distancia",
    values_to = "CV"
  )

ggplot(
  comparacion_larga,
  aes(x = p, y = CV, color = distancia)
) +
  geom_line() +
  geom_point() +
```

```
geom_hline(
  yintercept = 0.05,
  linetype = "dashed",
  color = "black"
) +
scale_x_log10() +
labs(
  title = "Comparación de concentración de distancias",
  x = "Dimensión (p)",
  y = "Coeficiente de variación",
  color = "Distancia"
) +
theme_minimal()
```

Warning: Removed 1 row containing missing values or values outside the scale range (`geom\_line()`).

Removed 1 row containing missing values or values outside the scale range (`geom\_point()`).



## 5. Ejercicio 5: PCA y reducción de dimensionalidad



```

# =====
# EJERCICIO 5: PCA Y REDUCCIÓN DE DIMENSIONALIDAD
# =====

library(tidyverse)

# -----
# 1. PREPARAR LOS DATOS
# -----

# Variables numéricas solicitadas
variables <- c("Age", "Fare", "SibSp", "Parch", "Pclass")

# Mantener únicamente filas completas
filas_completas <- complete.cases(titanic[, variables])

datos <- titanic[filas_completas, variables]

# Variable Survived para las mismas filas
survived <- titanic$Survived[filas_completas]

# -----
# 2. PCA
# -----

pca <- prcomp(
  datos,
  center = TRUE,
  scale. = TRUE
)

# Resumen del PCA
summary(pca)

```

Importance of components:

|                        | PC1    | PC2    | PC3    | PC4    | PC5     |
|------------------------|--------|--------|--------|--------|---------|
| Standard deviation     | 1.3196 | 1.2703 | 0.8313 | 0.7680 | 0.60330 |
| Proportion of Variance | 0.3483 | 0.3227 | 0.1382 | 0.1180 | 0.07279 |
| Cumulative Proportion  | 0.3483 | 0.6710 | 0.8092 | 0.9272 | 1.00000 |

```
# -----
# 3. VARIANZA EXPLICADA Y ACUMULADA
# -----

varianza <- pca$sdev^2 / sum(pca$sdev^2)

varianza_acumulada <- cumsum(varianza)

tabla_pca <- data.frame(
  Componente = 1:length(varianza),
  Varianza = varianza,
  Varianza_acumulada = varianza_acumulada
)

print(tabla_pca)
```

|   | Componente | Varianza   | Varianza_acumulada |
|---|------------|------------|--------------------|
| 1 | 1          | 0.34827175 | 0.3482717          |
| 2 | 2          | 0.32273262 | 0.6710044          |
| 3 | 3          | 0.13822498 | 0.8092294          |
| 4 | 4          | 0.11797728 | 0.9272066          |
| 5 | 5          | 0.07279337 | 1.0000000          |

```
# Número de componentes para 80% y 95%
componentes_80 <- which(varianza_acumulada >= 0.80)[1]
componentes_95 <- which(varianza_acumulada >= 0.95)[1]

cat("Componentes para >= 80%:", componentes_80, "\n")
```

Componentes para >= 80%: 3

```
cat("Componentes para >= 95%:", componentes_95, "\n")
```

Componentes para >= 95%: 5

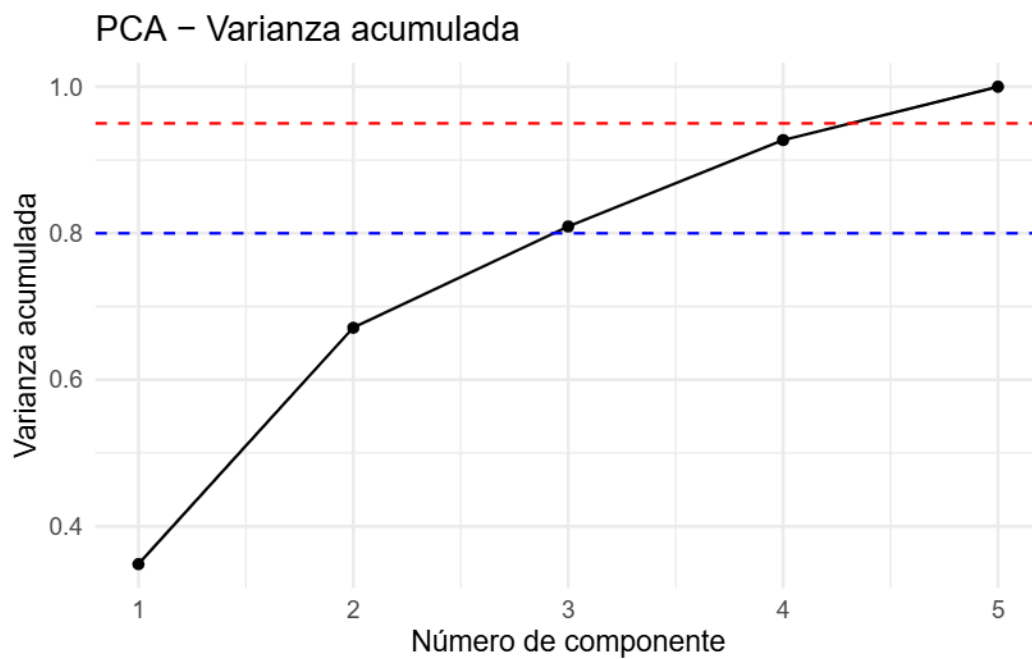
```
# -----
# 4. SCREE PLOT: VARIANZA ACUMULADA
# -----

ggplot(
```

```

tabla_pca,
aes(x = Componente, y = Varianza_acumulada)
) +
geom_line() +
geom_point() +
geom_hline(
  yintercept = 0.80,
  color = "blue",
  linetype = "dashed"
) +
geom_hline(
  yintercept = 0.95,
  color = "red",
  linetype = "dashed"
) +
labs(
  title = "PCA - Varianza acumulada",
  x = "Número de componente",
  y = "Varianza acumulada"
) +
theme_minimal()

```



```
# -----
# 5. LOADINGS DE PC1 Y PC2
# -----

loadings <- pca$rotation[, 1:2]

cat("\nLoadings de PC1 y PC2:\n")
```

Loadings de PC1 y PC2:

```
print(loadings)
```

|        | PC1        | PC2        |
|--------|------------|------------|
| Age    | 0.5459205  | -0.1971982 |
| Fare   | 0.4201282  | 0.5267194  |
| SibSp  | -0.3115174 | 0.5398679  |
| Parch  | -0.2058277 | 0.5630148  |
| Pclass | -0.6213332 | -0.2742927 |

```
# Importancia absoluta de las variables
cat("\nImportancia de variables en PC1:\n")
```

Importancia de variables en PC1:

```
print(sort(abs(loadings[, 1]), decreasing = TRUE))
```

| Pclass    | Age       | Fare      | SibSp     | Parch     |
|-----------|-----------|-----------|-----------|-----------|
| 0.6213332 | 0.5459205 | 0.4201282 | 0.3115174 | 0.2058277 |

```
cat("\nImportancia de variables en PC2:\n")
```

Importancia de variables en PC2:

```
print(sort(abs(loadings[, 2]), decreasing = TRUE))
```

| Parch     | SibSp     | Fare      | Pclass    | Age       |
|-----------|-----------|-----------|-----------|-----------|
| 0.5630148 | 0.5398679 | 0.5267194 | 0.2742927 | 0.1971982 |

```
# -----
# 6. K-MEANS SOBRE X ORIGINAL
# -----

# X escalada
X <- scale(datos)

set.seed(42)

kmeans_original <- kmeans(
  X,
  centers = 2,
  nstart = 25
)

cat("\nK-Means sobre X original:\n")
```

K-Means sobre X original:

```
tabla_original <- table(
  Cluster = kmeans_original$cluster,
  Survived = survived
)

print(tabla_original)
```

|         | Survived |     |
|---------|----------|-----|
| Cluster | 0        | 1   |
| 1       | 85       | 133 |
| 2       | 339      | 157 |

```

# -----
# 7. K-MEANS SOBRE LOS DOS PRIMEROS PCs
# -----

PC2 <- pca$x[, 1:2]

set.seed(42)

kmeans_pca <- kmeans(
  PC2,
  centers = 2,
  nstart = 25
)

cat("\nK-Means sobre los dos primeros PCs:\n")

```

K-Means sobre los dos primeros PCs:

```

tabla_pca_kmeans <- table(
  Cluster = kmeans_pca$cluster,
  Survived = survived
)

print(tabla_pca_kmeans)

```

|         | Survived |     |
|---------|----------|-----|
| Cluster | 0        | 1   |
| 1       | 81       | 131 |
| 2       | 343      | 159 |

```

# -----
# 8. FUNCIÓN PARA MEDIR COINCIDENCIA CON SURVIVED
# -----

accuracy_cluster <- function(cluster, y) {

  prediccion <- ave(
    y,
    cluster,

```

```

    FUN = function(x) {
      as.numeric(
        x == names(which.max(table(x)))
      )
    }
  )

  mean(prediccion)
}

# Coincidencia usando X original
accuracy_original <- accuracy_cluster(
  kmeans_original$cluster,
  survived
)

# Coincidencia usando los dos primeros PCs
accuracy_pca <- accuracy_cluster(
  kmeans_pca$cluster,
  survived
)

cat("\nCoincidencia X original:",
    round(accuracy_original * 100, 2), "%\n")

```

Coincidencia X original: 66.11 %

```

cat("Coincidencia 2 primeros PCs:",
    round(accuracy_pca * 100, 2), "%\n")

```

Coincidencia 2 primeros PCs: 66.39 %

```

# -----
# 9. COMPARACIÓN FINAL
# -----

if (accuracy_original > accuracy_pca) {

```

```
cat("\nLa representación X original presenta mayor coincidencia.\n")
} else if (accuracy_pca > accuracy_original) {
  cat("\nLa representación de los 2 primeros PCs presenta mayor coincidencia.\n")
} else {
  cat("\nAmbas representaciones presentan la misma coincidencia.\n")
}
```

La representación de los 2 primeros PCs presenta mayor coincidencia.

#### 1.0.4 Interpretación

El PCA se aplicó sobre Age, Fare, SibSp, Parch y Pclass, eliminando previamente las observaciones con valores faltantes y escalando las variables.

El número de componentes necesarios para alcanzar el 80% y el 95% de la varianza se obtiene directamente mediante `componentes_80` y `componentes_95`.

Los *loadings* permiten identificar qué variables tienen mayor influencia en PC1 y PC2. Las variables con mayor valor absoluto de loading son las que más contribuyen a cada componente.

Finalmente, se aplicó K-Means con  $k = 2$  tanto sobre las variables originales escaladas como sobre los dos primeros componentes principales. Las tablas de contingencia y el porcentaje de coincidencia permiten comparar qué representación separa mejor los grupos respecto a `Survived`.